

WFLOW-07: Problem-Triggered Remediation

Series: WFLOW — Workflows and Alert Notifications | **Notebook:** 7 of 10 | **Created:** January 2026 | **Last Updated:** 10/02/2026

Auto-Remediation with Workflows

Move beyond notifications to automated problem resolution. This notebook covers remediation patterns, safety guardrails, runbook automation, and common remediation scenarios.

Table of Contents

- 1. [Auto-Remediation Principles](#)
- 2. [Safety Guardrails](#)
- 3. [Common Remediation Patterns](#)
- 4. [Kubernetes Remediation](#)
- 5. [Cloud Resource Remediation](#)
- 6. [Runbook Automation](#)
- 7. [Approval Workflows](#)

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Platform subscription

Requirement	Details
Permissions	<code>automation:workflows:write</code>
Prior Knowledge	WFLOW-01 through WFLOW-06
External Systems	kubectl access, cloud API credentials

1. Auto-Remediation Principles

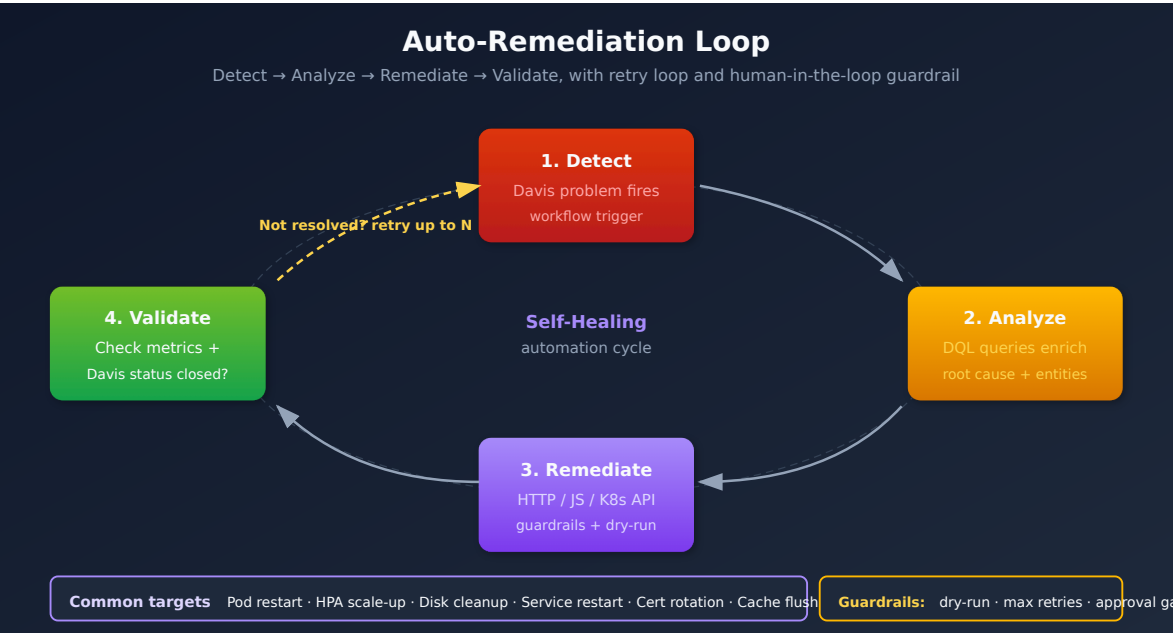
When to Automate Remediation

Scenario	Automate?	Reason
Pod crash loop	Yes	Well-defined fix (restart)
Disk space 90%	Yes	Clear action (cleanup)
Memory pressure	Maybe	Scale up vs investigate
Database deadlock	No	Needs investigation
Security incident	No	Requires human judgment

Remediation Maturity Model

Level 0: Manual remediation (runbook lookup)
 Level 1: Automated notification with runbook link
 Level 2: Semi-automated (human approval required)
 Level 3: Fully automated with guardrails
 Level 4: Self-healing with learning

Auto-Remediation Loop



2. Safety Guardrails

Essential Guardrails

Guardrail	Implementation	Purpose
Rate limiting	Max 3 remediations/hour	Prevent loops
Time window	Business hours only	Avoid off-hours changes
Environment scope	Non-prod first	Protect production
Cooldown period	30 min between actions	Allow stabilization
Rollback capability	Capture pre-state	Enable undo

Rate Limiting Implementation

```
import { queryExecutionClient } from '@dynatrace-sdk/client-query';

async function runQuery(query) {
  // queryExecute returns a requestToken instead of the result when the query
  // outlasts
  // requestTimeoutMilliseconds - poll until it reaches a final state (see
  // WFLOW-08 §2).
  const started = await queryExecutionClient.queryExecute({ body: { query,
    requestTimeoutMilliseconds: 30000 } });
  let r = started;
  while (r && (r.state === 'NOT_STARTED' || r.state === 'RUNNING')) {
    r = await queryExecutionClient.queryPoll({ requestToken:
      started.requestToken, requestTimeoutMilliseconds: 30000 });
  }
  if (!r || r.state !== 'SUCCEEDED') throw new Error(`DQL query did not
    succeed: ${r?.state ?? 'no response'}`);
  return r.result.records;
}

export default async function () {
  // Check recent remediation attempts. Workflow executions are recorded in
  // dt.system.events;
  // there is no automation.workflow.execution event type (a query on it
  // always returns 0,
  // so this guardrail would never trip).
  // runQuery() throws if the query does not complete, so a slow query stops
  // the
  // remediation instead of reading as "0 attempts" and sailing past the
  // limit.
  const records = await runQuery(`
    fetch dt.system.events, from:-1h
    | filter event.kind == "WORKFLOW_EVENT" and event.type ==
    "WORKFLOW_EXECUTION"
    | filter dt.automation_engine.state.is_final == true
    | filter contains(dt.automation_engine.workflow.title, "remediation")
    | summarize attempts = count()
  `);

  const attempts = Number(records[0]?.attempts ?? 0);
  const maxAttempts = 3;

  if (attempts >= maxAttempts) {
    return {
      proceed: false,
      reason: `Rate limit reached: ${attempts}/${maxAttempts} in last hour`
    };
  }
}
```

```
};  
}  
  
return { proceed: true, attempt_number: attempts + 1 };  
}
```

This counts remediation runs across all entities. Execution records do not carry the entity a run acted on, so a per-entity limit needs the entity ID recorded somewhere you can query — for example in a business event the remediation task emits.

Time Window Check

```
conditions:  
  - name: is_safe_window  
    expression: |  
      {% set hour = now().hour %}  
      {% set weekday = now().weekday() %}  
      {{ weekday < 5 and hour >= 6 and hour < 22 }}
```

```
tasks:  
  - name: remediation_action  
    conditions: [is_safe_window]  
    # ... action config
```

3. Common Remediation Patterns

Pattern 1: Service Restart

```
// Restart a service via SSH/API
import { execution } from '@dynatrace-sdk/automation-utils';
import { credentialVaultClient } from '@dynatrace-sdk/client-classic-environment-v2';

export default async function () {
  const event = (await execution()).params.event; // trigger payload
  const hostId = event.root_cause_entity_id;
  const serviceName = extractServiceName(event['event.name']);
  const token = (await credentialVaultClient.getCredentialsDetails({ id: 'CREDENTIALS_VAULT-XXXXXXXXXXXX' })).token;

  // Call external automation system
  const response = await fetch('https://automation.company.com/api/restart', {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${token}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      host: hostId,
      service: serviceName,
      reason: `Dynatrace problem ${event.display_id}`
    })
  });

  return {
    success: response.ok,
    message: await response.text()
  };
}
```

Pattern 2: Clear Cache

```
tasks:
  - name: clear_cache
    action: dynatrace.automations:http-function
    input:
      url: "https://{{ event().get('affected_entity_ids')[0]
}}.internal/admin/cache/clear"
      method: POST
      # Authentication: a Credential Vault token selected in the task's
      # Authentication field – never a static Authorization header
```

Pattern 3: Traffic Redirect

```
// Redirect traffic away from failing instance
import { execution } from '@dynatrace-sdk/automation-utils';

export default async function () {
  const event = (await execution()).params.event; // trigger payload
  const failingInstance = event.root_cause_entity_id;

  // Update load balancer to remove failing instance
  await updateLoadBalancer({
    action: 'drain',
    instance: failingInstance,
    reason: event.display_id
  });

  return { redirected: true };
}
```

4. Kubernetes Remediation

Pod Restart

```
import { KubeConfig, CoreV1Api } from '@kubernetes/client-node';
import { execution } from '@dynatrace-sdk/automation-utils';
import { credentialVaultClient } from '@dynatrace-sdk/client-classic-environment-v2';

export default async function () {
  const event = (await execution()).params.event; // trigger payload
  // Extract pod info from problem
  const podName = extractPodName(event.root_cause_entity_id);
  const namespace = extractNamespace(event.root_cause_entity_id);

  // Configure kubernetes client
  const kc = new KubeConfig();
  kc.loadFromString((await credentialVaultClient.getCredentialsDetails({ id:
'CREDENTIALS_VAULT-KUBECONFIG' })).token);
  const k8sApi = kc.makeApiClient(CoreV1Api);

  // Delete pod (Kubernetes will recreate it)
  try {
    await k8sApi.deleteNamespacedPod(podName, namespace);
    return {
      success: true,
      action: 'pod_restarted',
      pod: podName,
      namespace: namespace
    };
  } catch (error) {
    return {
      success: false,
      error: error.message
    };
  }
}
```


Deployment Rollback

```
import { KubeConfig, AppsV1Api } from '@kubernetes/client-node';
import { execution } from '@dynatrace-sdk/automation-utils';
import { credentialVaultClient } from '@dynatrace-sdk/client-classic-environment-v2';

export default async function () {
  const event = (await execution()).params.event; // trigger payload
  const deploymentName = extractDeployment(event);
  const namespace = extractNamespace(event);

  const kc = new KubeConfig();
  kc.loadFromString((await credentialVaultClient.getCredentialsDetails({ id: 'CREDENTIALS_VAULT-KUBECONFIG' })).token);
  const k8sApi = kc.makeApiClient(AppsV1Api);

  // Get deployment history
  const deployment = await k8sApi.readNamespacedDeployment(deploymentName, namespace);
  const currentRevision = deployment.body.metadata.annotations['deployment.kubernetes.io/revision'];

  // Rollback to previous revision
  await k8sApi.patchNamespacedDeployment(
    deploymentName,
    namespace,
    {
      spec: {
        rollbackTo: { revision: parseInt(currentRevision) - 1 }
      }
    },
    undefined,
    undefined,
    undefined,
    undefined,
    { headers: { 'Content-Type': 'application/strategic-merge-patch+json' } }
  );

  return {
    action: 'rollback',
    deployment: deploymentName,
    from_revision: currentRevision,
    to_revision: parseInt(currentRevision) - 1
  };
}
```

Horizontal Pod Autoscaler Adjustment

```
import { credentialVaultClient } from '@dynatrace-sdk/client-classic-environment-v2';

const K8S_API_SERVER = 'https://k8s-api.example.com'; // add to Settings
> External requests

export default async function () {
  const k8sToken = (await credentialVaultClient.getCredentialsDetails({ id:
'CREDENTIALS_VAULT-XXXXXXXXXXXX' })).token;
  // Temporarily increase replicas for high load
  const response = await fetch(

`${K8S_API_SERVER}/apis/autoscaling/v2/namespaces/production/horizontalpodauto
scalers/checkout-hpa`,
    {
      method: 'PATCH',
      headers: {
        'Authorization': `Bearer ${k8sToken}`,
        'Content-Type': 'application/merge-patch+json'
      },
      body: JSON.stringify({
        spec: {
          minReplicas: 5 // Increase from default 2
        }
      })
    }
  );

  return { scaled: response.ok };
}
```

5. Cloud Resource Remediation

AWS EC2 Instance Restart

```
import { EC2Client, RebootInstancesCommand } from '@aws-sdk/client-ec2';
import { execution } from '@dynatrace-sdk/automation-utils';
import { credentialVaultClient } from '@dynatrace-sdk/client-classic-environment-v2';

export default async function () {
  const event = (await execution()).params.event; // trigger payload
  // A username/password credential holding the access key ID and secret
  const aws = await credentialVaultClient.getCredentialsDetails({ id:
'CREDENTIALS_VAULT-XXXXXXXXXXXX' });
  const ec2Client = new EC2Client({
    region: 'us-east-1',
    credentials: {
      accessKeyId: aws.username,
      secretAccessKey: aws.password
    }
  });

  const instanceId = extractEC2InstanceId(event.root_cause_entity_id);

  const command = new RebootInstancesCommand({
    InstanceIds: [instanceId]
  });

  await ec2Client.send(command);

  return {
    action: 'ec2_reboot',
    instance_id: instanceId
  };
}
```

AWS Lambda Function Redeploy

```
import { LambdaClient, UpdateFunctionCodeCommand } from '@aws-sdk/client-lambda';
import { execution } from '@dynatrace-sdk/automation-utils';

const LAMBDA_BUCKET = 'my-lambda-artifacts';

export default async function () {
  const event = (await execution()).params.event; // trigger payload
  const lambdaClient = new LambdaClient({ region: 'us-east-1' });
  const functionName = extractLambdaName(event);

  // Force cold start by updating function
  const command = new UpdateFunctionCodeCommand({
    FunctionName: functionName,
    S3Bucket: LAMBDA_BUCKET,
    S3Key: `${functionName}/latest.zip`
  });

  await lambdaClient.send(command);

  return { redeployed: functionName };
}
```

Azure App Service Restart

```
import { credentialVaultClient } from '@dynatrace-sdk/client-classic-environment-v2';

const AZURE_SUBSCRIPTION_ID = '&lt;subscription-id&gt;';

export default async function () {
  const azureToken = (await credentialVaultClient.getCredentialsDetails({ id: 'CREDENTIALS_VAULT-XXXXXXXXXXXX' })).token;
  const response = await fetch(

`https://management.azure.com/subscriptions/${AZURE_SUBSCRIPTION_ID}/resourceGroups/${resourceGroup}/providers/Microsoft.Web/sites/${appName}/restart?api-version=2022-03-01`,
    {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${azureToken}`
      }
    }
  );

  return { restarted: response.ok };
}
```

6. Runbook Automation

Runbook Lookup and Execution

```
import { execution } from '@dynatrace-sdk/automation-utils';

export default async function () {
  const event = (await execution()).params.event; // trigger payload
  // Map problem types to runbooks
  const runbookMap = {
    'High CPU': 'runbook-cpu-investigation',
    'Memory exhaustion': 'runbook-memory-cleanup',
    'Disk space': 'runbook-disk-cleanup',
    'Connection pool': 'runbook-connection-reset'
  };

  // Find matching runbook
  let runbookId = null;
  for (const [pattern, id] of Object.entries(runbookMap)) {
    if (event['event.name'].includes(pattern)) {
      runbookId = id;
      break;
    }
  }

  if (!runbookId) {
    return {
      action: 'no_runbook',
      message: 'No automated runbook for this problem type'
    };
  }

  // Execute runbook via automation system
  const result = await executeRunbook(runbookId, {
    problem_id: event.display_id,
    entity_id: event.root_cause_entity_id
  });

  return {
    action: 'runbook_executed',
    runbook: runbookId,
    result: result
  };
}
```

Include Runbook Link in Notification

```
message: |
  :warning: *{{ event()['event.name'] }}*

  *Suggested Runbook:*
  {% if "CPU" in event()["event.name"] %}
  &lt;https://wiki.company.com/runbooks/cpu-investigation|CPU Investigation
  Runbook&gt;
  {% elif "Memory" in event()["event.name"] %}
  &lt;https://wiki.company.com/runbooks/memory-troubleshooting|Memory
  Troubleshooting Runbook&gt;
  {% else %}
  &lt;https://wiki.company.com/runbooks|Browse Runbooks&gt;
  {% endif %}
```

7. Approval Workflows

Human-in-the-Loop Pattern

The Slack Connector's **Request approval** action is the built-in way to pause for a person. The docs describe it as: *"Send an approval request to a Slack channel and pause the workflow until a response is received."* The action posts the message with its own **Approve** and **Decline** buttons. It resumes the workflow when someone answers or the timeout set in its **Options** tab passes, and replies in the thread with the outcome.

```

tasks:
  # 1. Ask for approval and pause until someone answers or the timeout passes
  - name: request_approval
    action: dynatrace.slack:request-approval
    timeout: 1800          # 30 minutes, in seconds (Options tab)
    input:
      connection: slack-production
      channel: "#approvals"
      message: "*Remediation approval required*\nProblem: {{ event()
['event.name'] }}\nProposed action: restart pod\nEntity: {{ event()
['root_cause_entity_id'] }}"

  # 2. Execute only after the approval task finished successfully
  - name: execute_remediation
    action: dynatrace.automations:run-javascript
    predecessors: [request_approval]
    conditions:
      states:
        request_approval: SUCCESS
    input:
      script: |
        // Perform remediation action

```

Test Decline before you rely on the gate. The docs list the action's outputs (`approvalType` plus the channel, message and thread IDs and a permalink), but they name no approved/declined field and do not say which task state a Decline or a timeout leaves. Run one approval, one decline and one timeout in a test workflow, and read the task's state and result each time. The same test confirms that the task timeout in the YAML (set in the Options tab as **Adapt timeout**) is the approval window. If a Decline also ends in `SUCCESS`, gate on whatever field distinguishes it, or the remediation runs either way. The message takes Slack Markdown only, so do not add buttons of your own.

Sources: [Slack Connector actions \(DT docs\)](#) — *"When an approver responds, or the configured timeout is reached, the workflow resumes and posts the outcome as a reply in the original message thread."*, *"Configure the timeout in the action's Options tab."*

Auto-Approve for Non-Production

```
conditions:
  - name: is_production
    expression: '{{ "Production" in event().get("management_zones", []) }}'

tasks:
  # Auto-execute for non-prod
  - name: auto_remediate
    conditions: [not is_production]
    # ... remediation action

  # Require approval for prod
  - name: request_prod_approval
    conditions: [is_production]
    # ... approval workflow
```

Monitor Remediation Workflows

```
// Remediation workflow success rates
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`,
// and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in
// any spelling – those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
// `dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by
// `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION ·
//   WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type – e.g. run-javascript,
//   send-email,
//                                       snow-search-incidents)
//   .state                             (was task.status / execution.status)
//                                       values RUNNING · SUCCESS · ERROR ·
//   DISCARDED · SKIPPED
//                                       – note "FAILED" is NOT a value; it
//   is ERROR
//   .state.is_final                    true only on terminal records –
//   filter on it, otherwise a
//                                       single execution is counted once as
//   RUNNING and again as
//                                       SUCCESS/ERROR
//   .state_info                        (was task.error) · duration (was
//   task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT"
//   | limit 1
//   fetch dt.system.events, from:-7d
//   | filter event.kind == "WORKFLOW_EVENT"
//   | filter event.type == "WORKFLOW_EXECUTION"
//   | filter dt.automation_engine.state.is_final == true
//   | summarize {total = count(), succeeded = countIf(dt.automation_engine.state
//   == "SUCCESS")}, by:{dt.automation_engine.workflow.title}
//   | fieldsAdd success_pct = round(succeeded * 100.0 / total, decimals: 1)
//   | sort total desc
//   | limit 20
```

```
// Distinct problems closed per day
// Counted with countDistinctExact(display_id) since 09/24/2026: `events`
holds several records per
// closed problem, so count() over-reported closed problems ~4x on the
validation tenant.
// Field names corrected 08/12/2026: on `events`, a Davis problem carries
`event.status` and
// `event.category` – there are no bare `status` / `severity` fields, so those
filters matched
// nothing. `event.status` values are ACTIVE / CLOSED – "OPEN" is not one of
them.
fetch events, from:-30d
| filter event.kind == "DAVIS_PROBLEM" and event.status == "CLOSED"
| summarize total_closed = countDistinctExact(display_id), by:{time_bucket =
bin(timestamp, 24h)}
| sort time_bucket asc
```

Next Steps

With remediation patterns in place, learn advanced integrations:

Recommended Path

1. **WFLOW-08: JavaScript & HTTP Actions** - Custom integrations
2. **WFLOW-09: Security & Governance** - Production best practices

Key Takeaways

- **Guardrails** prevent runaway automation
 - **Rate limiting** avoids remediation loops
 - **Kubernetes** can be automated via API
 - **Runbook mapping** ties problems to solutions
 - **Approval workflows** for production changes
-

Summary

In this notebook, you learned:

- Auto-remediation principles and maturity model
 - Safety guardrails (rate limiting, time windows)
 - Common remediation patterns
 - Kubernetes pod/deployment remediation
 - Cloud resource remediation (AWS, Azure)
 - Runbook automation integration
 - Human-in-the-loop approval workflows
-

References

- [Workflow actions umbrella \(DT docs\)](#)
 - [Run JavaScript action \(DT docs\)](#)
 - [HTTP request action \(DT docs\)](#)
 - [Workflow reference / Jinja expressions \(DT docs\)](#)
 - [Davis Problems app \(DT docs\)](#)
 - [Kubernetes API reference \(kubernetes.io\)](#)
-

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.